

Aufgabe 6

Aufgabe 6.1

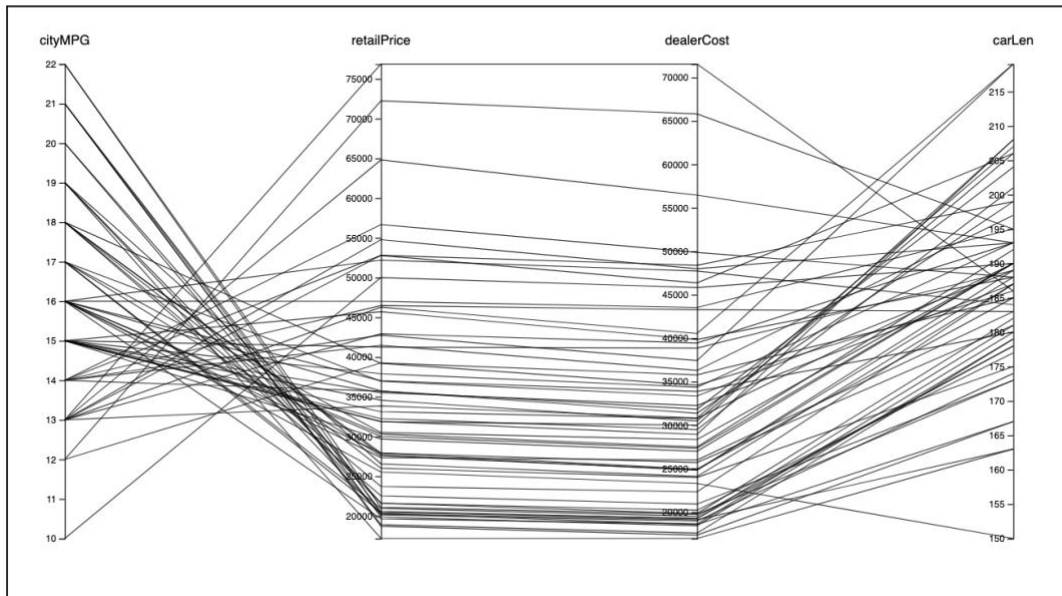
Erklärung: Das Programm verwendet die cars-Liste aus Übung 2 als mehrdimensionale Tabelle und stellt eine ausgewählte Teilmenge als Parallel Coordinates Plot dar. Im Mittelpunkt steht das Filtern und Umformen der Rohdaten in eine generische Repräsentation: Mit filterCars werden nur Fahrzeuge vom Typ SUV behalten, bei denen die vier interessierenden Felder cityMPG, retailPrice, dealerCost und carLen nicht Nothing sind; anschließend wandelt carsToMultiDimData jeden gefilterten Car mit Maybe.map4 in einen MultiDimPoint um, dessen value-Liste genau diese vier Werte (als Float) in fester Reihenfolge enthält, und verpackt das Ergebnis zusammen mit dimDescription = ["cityMPG", "retailPrice", "dealerCost", "carLen"] in einen MultiDimData-Record. Die Funktion parallelCoordinatesPlot war model zeichnet diesen generischen Datentyp dimensionsunabhängig: Die Höhe ergibt sich aus $h = w / ar$, die x-Position der Achsen wird über Scale.linear (0, w) (1, toFloat numDims) aus dem Achsindex berechnet, und für jede Dimension wird aus List.minimum/List.maximum der jeweiligen Spalte ein eigenes y-Scale.linear (h, 0) (min, max) abgeleitet, sodass jede Achse ihren vollen Datenbereich nutzt. Die vertikalen Achsen werden mit Axis.left und einem text_-Label pro Dimension via List.indexedMap platziert, und jeder Datenpunkt wird mit List.indexedMap über seine value-Liste in eine Folge von (xPos, yPos)-Punkten übersetzt, die Shape.line Shape.linearCurve und Path.element als zusammenhängenden SVG-Pfad rendern. Im View wird zusätzlich die Anzahl der gefilterten SUVs ausgegeben und der Plot in einem schwarz umrandeten div dargestellt — so wird die Korrelationsstruktur zwischen Verbrauch, Preisen und Fahrzeuglänge der SUVs durch das parallele Achsenraster und die verbindenden Polylinien direkt visuell erfassbar.

Screenshot auf der nächsten Seite

Screenshot:

SUVs

- Number of filtered Cars: 59



Link: <https://ellie-app.com/yHx4RCQPvgna1>

Aufgabe 6.2

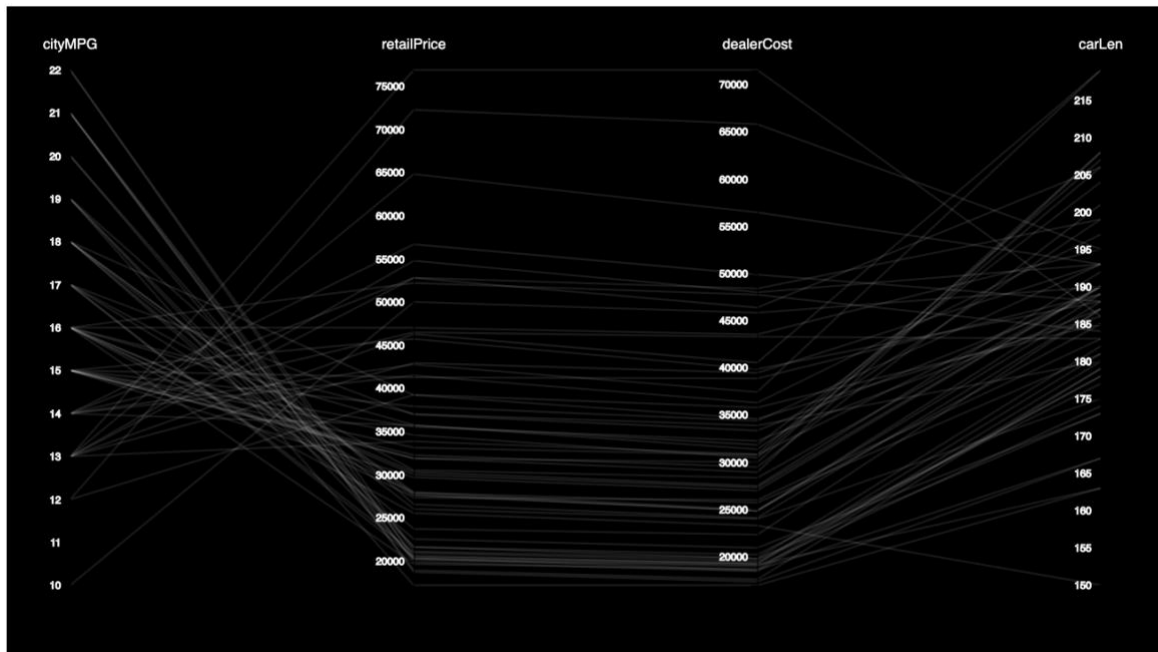
Erklärung:

Die Implementierung baut direkt auf 6.1 auf — die gesamte Logik der parallelen Koordinaten (Skalen, Achsen, Linienpfade, SUV-Filter, vier Dimensionen cityMPG, retailPrice, dealerCost, carLen) bleibt unverändert; geändert wurde ausschließlich die visuelle Darstellung. Im Plot wurde vor den Linien ein `TypedSvg.rect` mit `width = w`, `height = h` und `TA.fill (Paint Color.black)` als umschließendes Rechteck eingefügt, wodurch die gesamte Plot-Fläche schwarz wird. Die Polygonzüge der Datensätze werden nun mit `lineColor = Color.rgba 1 1 1 0.1` (weiß mit Deckkraft 10 %) und `strokeWidth 2` gezeichnet. Da viele halbtransparente Linien überlagernd auf dem dunklen Hintergrund liegen, summiert sich die Helligkeit dort, wo viele Liniensegmente einander schneiden oder dicht beieinander verlaufen — diese Bereiche erscheinen heller, während Einzellinien fast unsichtbar bleiben. Dieser Additionseffekt entspricht genau der X-Ray-Technik aus den Folien und macht häufige Schnittpunkte (also Korrelationsmuster zwischen den Dimensionen) als leuchtende Strukturen sichtbar. Damit Achsen und Beschriftungen auf dem schwarzen Hintergrund lesbar bleiben, wurden zusätzlich `stroke` und `fill` der Achsengruppen auf weiß gesetzt.

Screenshot:

SUVs (X-Ray)

- Number of filtered Cars: 59



Link: <https://ellie-app.com/yHBRNLWfVva1>

Aufgabe 6.3

Erklärung:

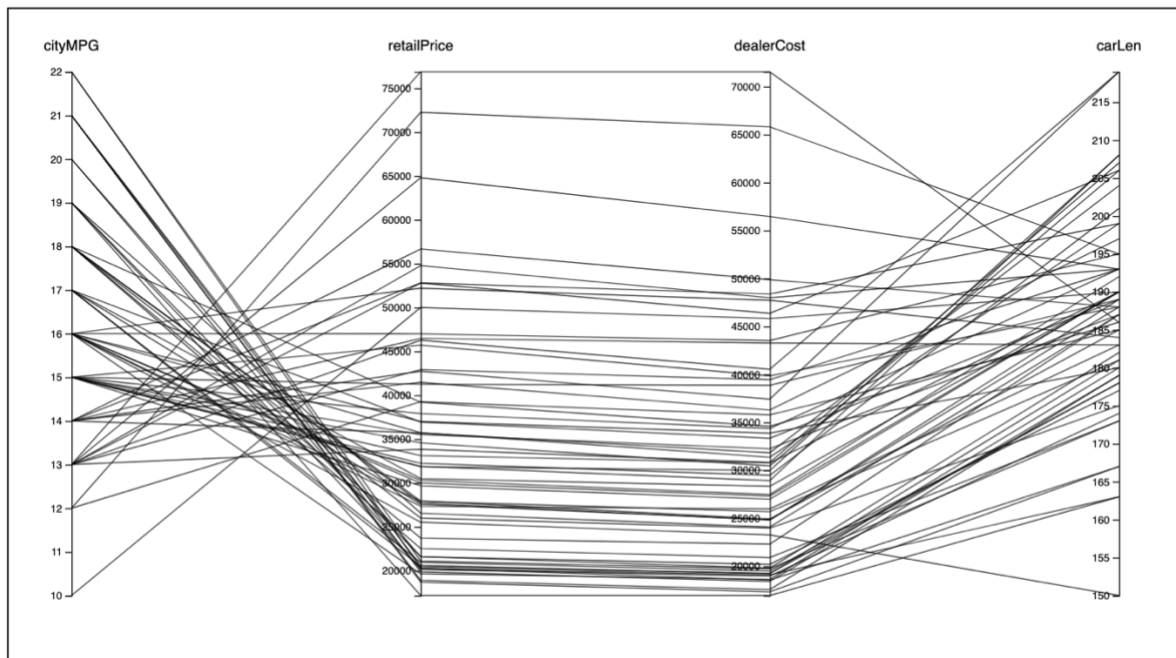
Die Idee dieser Aufgabe ist es, den Code aus 6.1 so umzubauen, dass die Reihenfolge der Dimensionen nicht nur mit einer einzigen Code-Änderung, sondern sogar zur Laufzeit per Knopfdruck gewechselt werden kann, ohne die Plot-Logik anzufassen. Dafür wandert die zentrale Liste `dimensions : List ( String, FilteredCar -> Float )` aus 6.1 in das Modell einer `Browser.element-App`: `Model = { dimensions : List Dimension, filtered : List FilteredCar }`. Jeder Eintrag bündelt weiterhin Beschriftung und Zugriffsfunktion (z. B. (`"cityMPG", .cityMPG >> toFloat`)), und `carsToFiltered` läuft einmalig in `init`, sodass im heißen Pfad nur noch reine `FilteredCar -> Float`-Zugriffe stattfinden. Die Interaktion läuft über `Msg = MoveUp Int | MoveDown Int`: jede Achse wird im View als `<li>` mit Name und zwei Buttons („up“, „down“) gerendert, und ein Klick schickt den Listenindex an `update`. Dort übernimmt eine selbst geschriebene `swapAt i j list` die Arbeit — sie holt die beiden Werte mit `List.drop k |> List.head`, prüft per `case` beide Indizes und tauscht sie via `List.indexedMap`. Bewusst ohne `elm-community/list-extra`, damit Ellie ohne Zusatzpakete läuft. `update` macht zusätzlich Bounds-Checks (`MoveUp` nur bei `i > 0`,

MoveDown nur bei $i < \text{length} - 1$), sodass Buttons am Rand wirkungslos bleiben. Damit der Plot beim Klick aktualisiert, bekommt filteredToMultiDimData die Dimensionen jetzt als Parameter statt als Top-Level-Konstante; intern bleibt alles gleich (accessors = List.map Tuple.second dims, labels = List.map Tuple.first dims, value = List.map (\f -> f c) accessors), sodass die Reihenfolge der Achsen, der Beschriftungen und der Float-Werte garantiert synchron bleibt. parallelCoordinatesPlot selbst weiß nichts von alldem — er iteriert weiterhin mit numDims = List.length model.dimDescription und List.indexedMap über das, was er bekommt, und bleibt damit komplett unverändert gegenüber 6.1. Hinzufügen oder Entfernen einer Dimension bedeutet weiterhin nur einen Listeneintrag in initialDimensions (plus passendes Feld im FilteredCar); das Umsortieren ist zur Compile-Zeit eine Listen-Umordnung und zur Laufzeit eine Folge von swapAt-Aufrufen — die Aufgabenstellung damit doppelt erfüllt.

Screenshots:

SUVs

- Number of filtered Cars: 59
- cityMPG
- retailPrice
- dealerCost
- carLen



Link: <https://ellie-app.com/yHChNgjvSQFa1>